

计算机软件技术基础

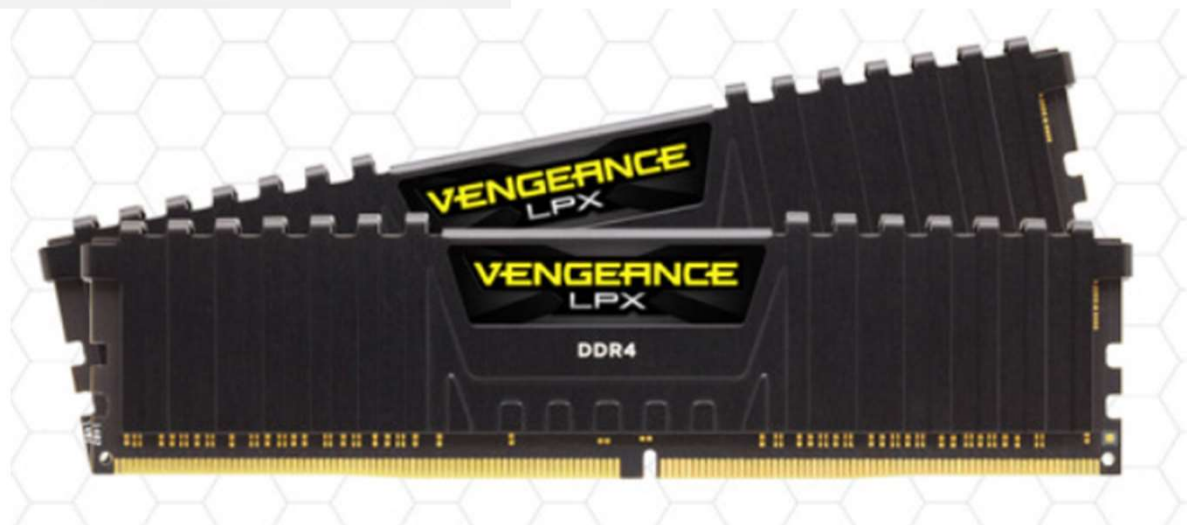
OS-2

2024 年 4 月



信息科学与工程学院
COLLEGE OF INFORMATION SCIENCE AND ENGINEERING

4.2 存储管理



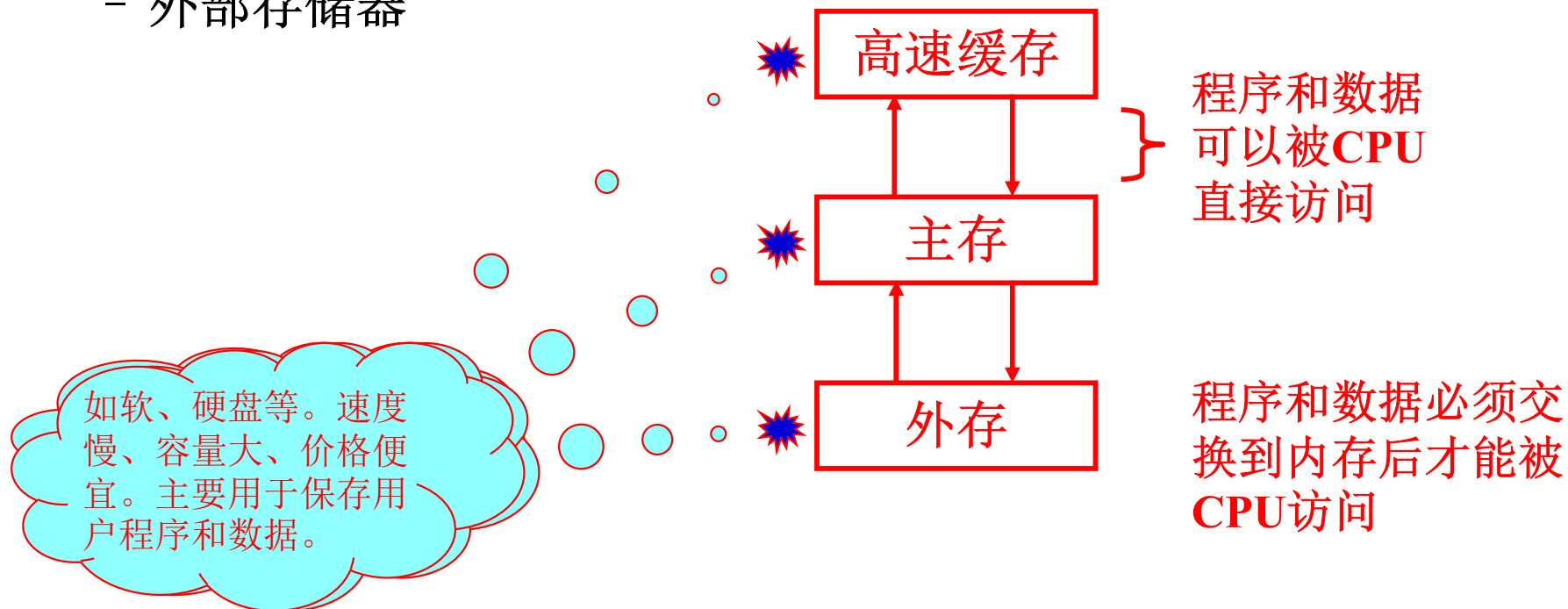
4.2 存储管理

- 存储（memory）管理
 - 是指主存储器（内存）资源的管理
 - 管理如何分配内存给作业，用以存储数据和相应的程序
 - 主要解决多道程序在内存如何分配而互不冲突、不被破坏的问题
- 存储管理的分类
 - **实存储管理**：对实际存在的存储器硬件进行管理
 - **虚拟存储管理**：把外存储器的一部分和实际的存储器放在一起进行管理，实现存储空间的扩大

4.2 存储管理

- 存储器的分级结构

- 高速缓冲存储器
- 主存储器（内存）
- 外部存储器



4.2 存储管理

- 存储管理的功能
 - 内存分配
 - 算法：
 - 分配给谁
 - 分配多少
 - 分在何处
 - 数据结构：
 - 存储器状态
 - 分配的状态
 - 等

4.2 存储管理

- 存储管理的功能

- 地址转换或重定位

- 名空间:

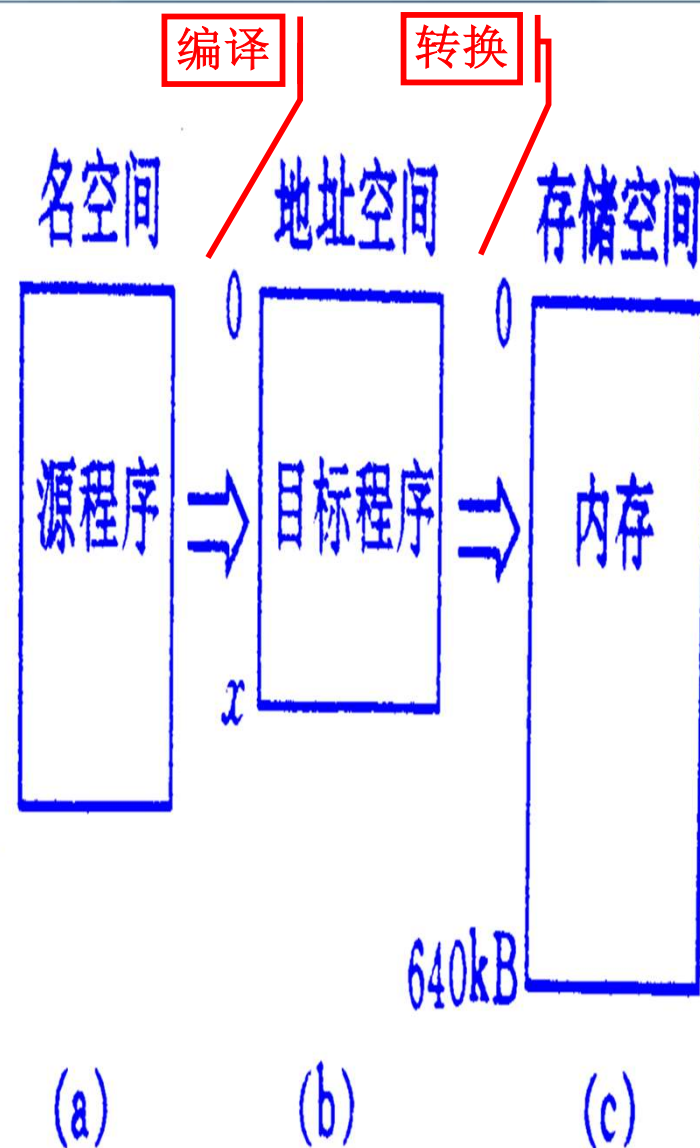
- 源程序(符号、指令、数据等)

- 地址空间:

- 程序或数据的地址（逻辑地址、相对地址）

- 存储空间:

- 存储器的地址（物理地址、绝对地址）



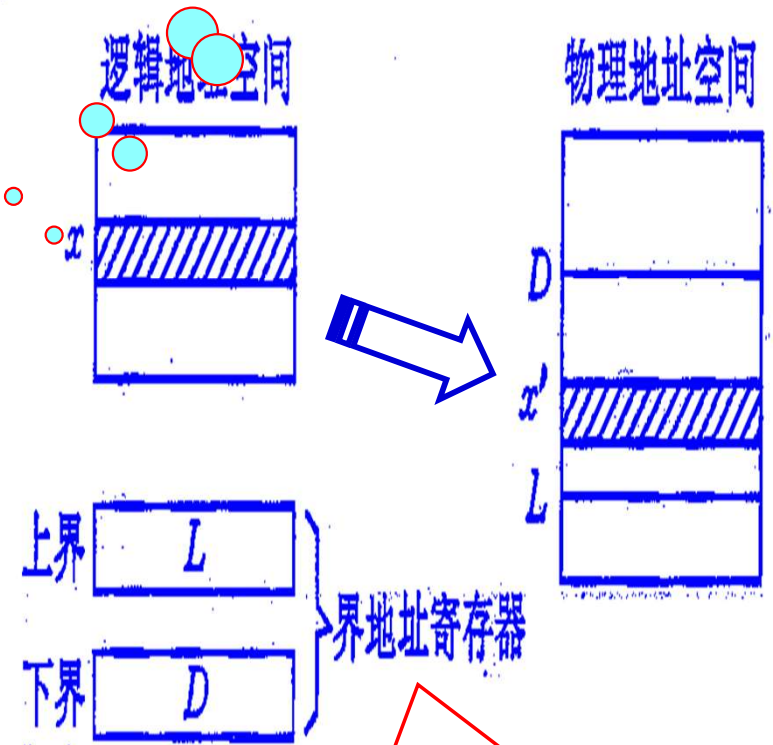
4.2 存储管理

- 存储管理的功能

- 地址转换或重定位

- 地址转换/重定位：就是把相对地址转换为绝对地址，包括程序中与地址有关的指令的修改
 - 静态重定位：装入程序时进行地址转换
 - 动态重定位：在程序执行过程中进行地址的转换，CPU执行程序指令时由动态变换机构自动进行地址转换

程序被原封不动地放入内存，只有在执行时地址才被转换。程序可以在内存中浮动



保存可分配物理地址空间的上下界

4.2 存储管理

- 存储管理的功能

- 栈(Stack) vs 堆(Heap)

- 都是程序运行时占用的内存

- 每个线程(thread)有自己的栈(Stack)

- MyClass my_obj; //! In a function

- 函数结束 自动释放内存

- 一般大小固定(依赖于语言、编译器)

- 耗尽栈(Stack): stack overflow, 例: 嵌套函数

4.2 存储管理

- 存储管理的功能

- 栈(Stack) vs 堆(Heap)

- 不同线程(threads) **共享**堆(Heap)

- MyClass *p_my_obj = new MyClass(); //! Somewhere

- delete m; //! Somewhere

- **编程者负责释放内存: free, delete, or delete[]**

- » 自动收集内存: Java

- 堆(Heap)大小可以变化, 可以从OS申请

- Fragmentation: 内存碎片化



4.2 存储管理

- 存储管理的功能
 - 栈(Stack) vs 堆(Heap)
 - 栈(Stack): 快
 - 一般事前可预判所需内存大小
 - 使用较小量内存
 - 堆(Heap): 慢
 - 所需内存大小不可预判
 - 使用较大量内存

4.2 存储管理

- 存储管理的功能

- 存储保护

- 多道程序时，保护作业程序之间的存储区不被恶意的访问所干扰和破坏
 - 对并发性和共享性尤为重要
 - 静态重定位下，可以采用界地址寄存器来实现

- $D \leq X' < L$

- 一旦越界，则产生中断，转向中断处理程序



4.2 存储管理

- 存储管理的功能

- 内存扩充

- 当作业程序的地址空间 $>$ 存储空间时，必须采用内存扩充技术，联合内外存，为用户提供更大的存储空间

- 内存扩充技术：

- 覆盖、交换、虚拟存储

4.2 存储管理

- 实存储管理

- 作业程序调入内存时，分配一个不小于作业地址空间的连续存储空间给该作业程序
 - 足够作业地址空间
 - 在物理内存内的“连续存储空间”
- 存储空间不够时：
 - 采用“覆盖”或“交换”技术 ➔ 内存扩充

4.2 存储管理

- 实存储管理

- 1) 分区分配

- 最简单的分配方法，适用于小型、微型机的多道程序
 - 内存以区划分
 - 每个分区分配给一个作业程序
 - 用静态重定位方式进行地址转换
 - 固定分区分配，按事先划分好的分区为单位进行分配
 - 可变分区分配，不事先划分好分区，而按需分配内存块给作业程序
 - 空间分配，“占用块”和“空闲块”



4.2 存储管理

- 实存储管理

- 1) 分区分配 —— 固定分区分配（又称静态分配）

- 数据结构:

- 线性表: 保存 分区号、容量、起始地址、分配状态等

- 优点:

- 算法简单

- 缺点:

- 空间浪费

- 内存碎片

4.2 存储管理

• 实存储管理

1) 分区分配 —— 固定分区分配（又称静态分配）

- **分配**：当作业调入内存时，根据作业大小，在表中查找满足要求且未分配的分区，找到后进行分配并修改分配状态为“已分配”
- **释放/去分配**：当作业完成时，查找相应区号，修改其状态为“未用”

				内 存	
区号	容量	起始位置	状态		
1	8kB	312kB	已分配	312kB	OS
2	32kB	320kB	已分配	320kB	(8kB)
3	32kB	352kB	未 用	352kB	(32kB)
4	120kB	384kB	未 用	384kB	(32kB)
5	520kB	504kB	已分配	504kB	(120kB)
				1024kB	(520kB)



4.2 存储管理

- 实存储管理

- 1) 分区分配 -- 可变分区分配（动态分配）

- 数据结构:

- 双向链表：表示未分配的“空闲块”链

- » 结点含义如图所示。

- 也可以采用顺序存储的线性表，表示已分分区和空白分区（空闲块）

4.2 存储管理

- 实存储管理

- 1) 分区分配 -- 可变分区分配 (动态分配)

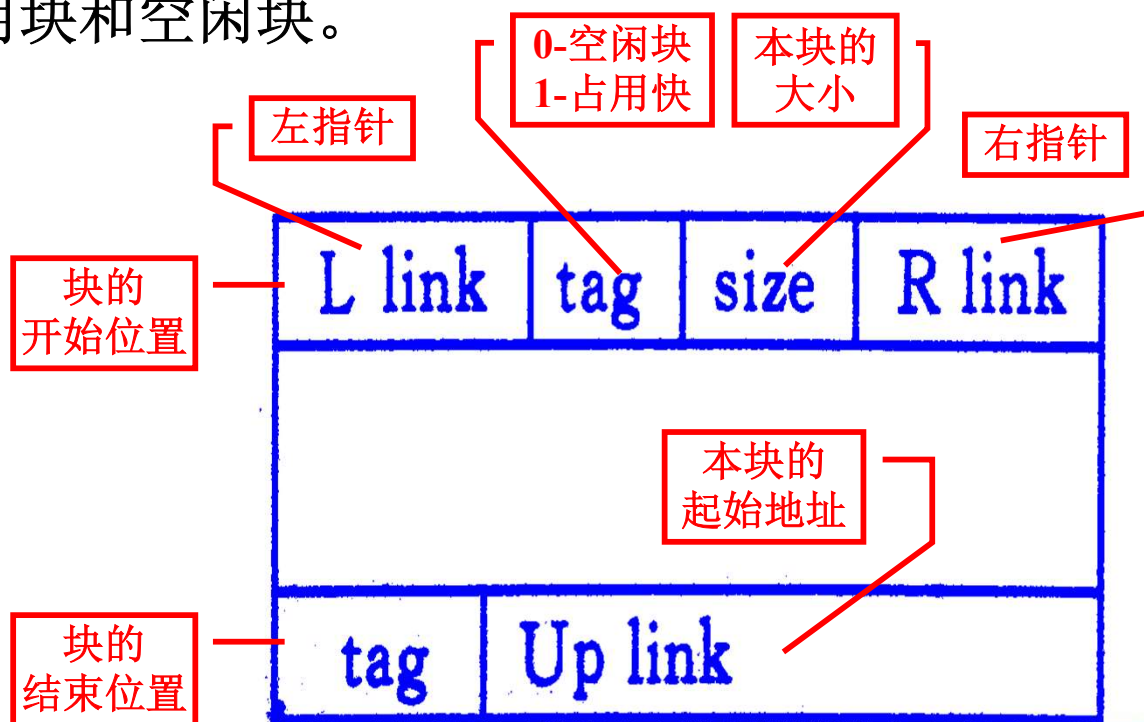
- 双向链表结点含义
 - 初始时，整个用户区只有一个空闲块。随着作业的不断进入和退出，会形成很多占用块和空闲块。



(a)



(b)



4.2 存储管理

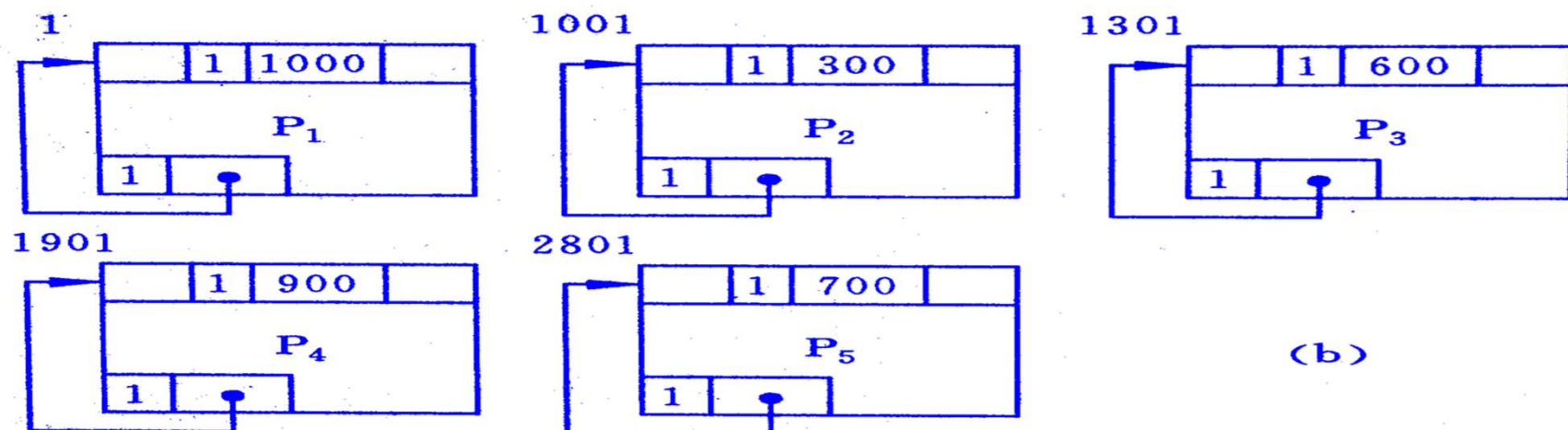
- 实存储管理

- 1) 分区分配 -- 可变分区分配 (动态分配)

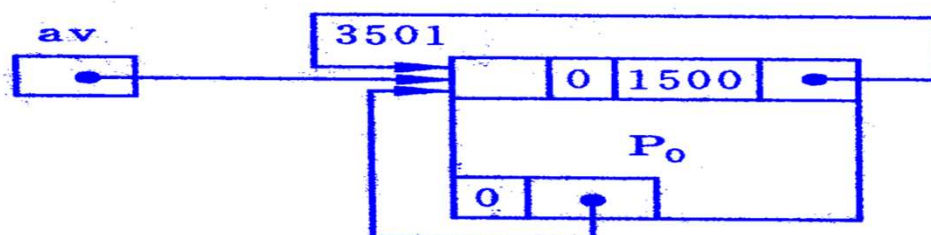
分配: 查找满足要求的空闲块进行分配, 并修改占用块及空闲块的信息



(a)



(b)



(c)

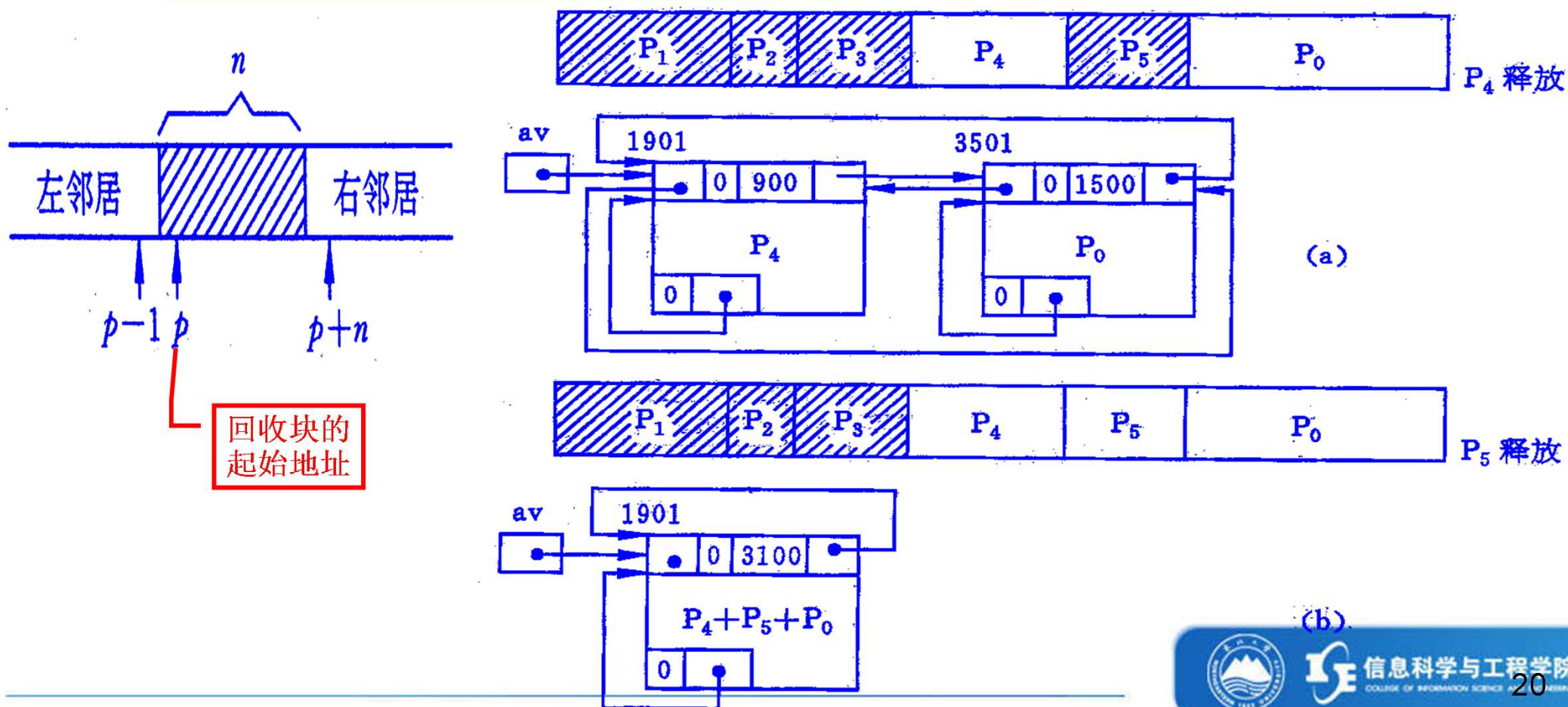
4.2 存储管理

- 实存储管理

- 1) 分区分配 -- 可变分区分配 (动态分配)

释放/去分配: 作业完成后, 收回空间, 插入到空闲块链表当中。
(需要查找插入位置)

判断左右邻接关系, 合并相邻空闲区域



4.2 存储管理

- 实存储管理

- 1) 分区分配

- 空闲块分配算法(存储分配策略): 有多个空闲块满足作业大小时, 如何进行分配的问题

- “首次”适应算法, 从链表头开始, 找第一个适合的空闲块

- » 优点: 简单、快

- » 缺点: 可能毁掉一个大空白分区, 而无法分配大作业

4.2 存储管理

- 实存储管理

- 1) 分区分配

- 空闲块分配算法(存储分配策略): 有多个空闲块满足作业大小时, 如何进行分配的问题

- “最佳”适应算法, 找大小最接近的空闲块

- » 优点: 不会毁掉大分区

- » 缺点: 需要排序空闲块、会出现内存“碎片”

4.2 存储管理

- 实存储管理

- 1) 分区分配

- 空闲块分配算法(存储分配策略): 有多个空闲块满足作业大小时, 如何进行分配的问题

- “最差”适应算法, 找最大的空闲块

- » 优点: 剩余块可以再分

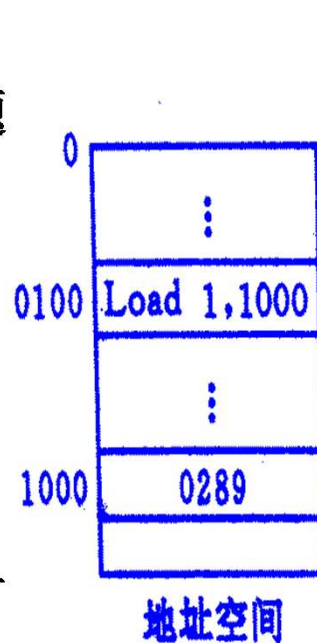
- » 缺点: 需要排序

4.2 存储管理

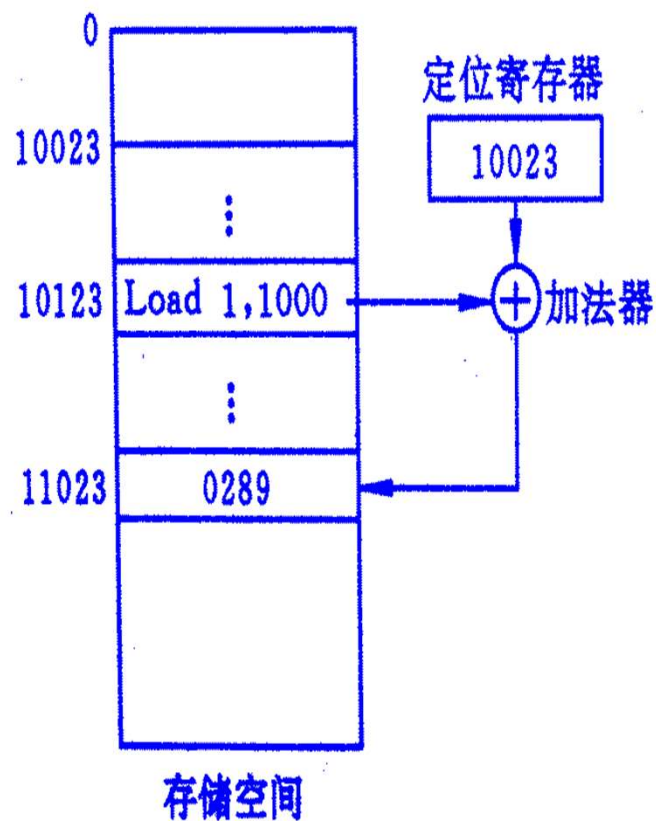
• 实存储管理

2) 可重定位分区分配

- 动态重定位+分区分配
- 出发点:
 - 解决分区分配中的碎片问题
- 方法:
 - 采用存储紧缩技术
- 紧缩时机:
 - 当分区释放时
 - 当找不到大小合适的分区时
- 不足:
 - 要有硬件支持,
 - 紧缩过程需要时间



(a)



(b)

4.2 存储管理

• 实存储管理

3) 覆盖技术

- 当作业程序的地址空间 $>$ 主存可用空间时

- 作业程序须满足树状的模块结构

- 根段，主程序
- 覆盖段，同层模块
- 同层模块，逻辑独立

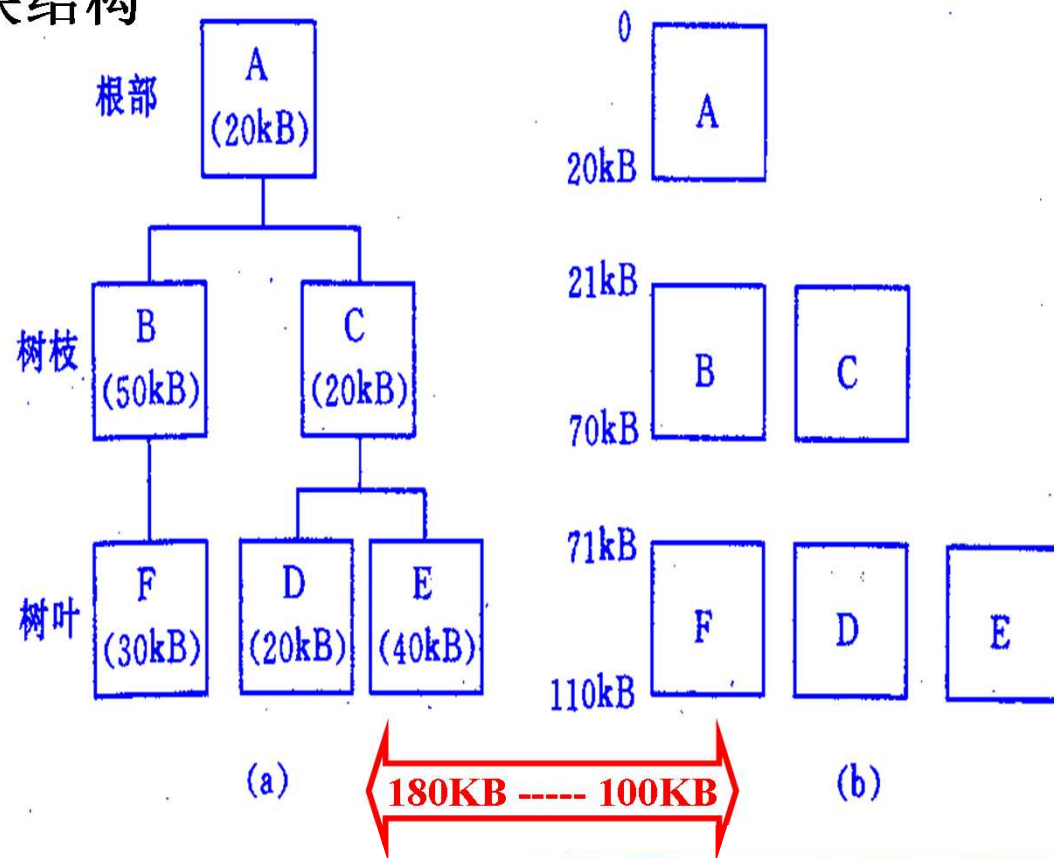
- 按覆盖段分配内存

- 覆盖描述语言

- ROOT A-(B-F, C-(D, F));

- END

- 应用逐渐减少



4.2 存储管理

- 实存储管理

- 4) 交换技术

- 用于解决内存不足
 - 只允许一个或几个用户作业保留在主存中
 - 让出CPU使用权的同时，退出内存
 - 内存不够用时，让一些作业程序退出内存
 - “退出”是指将内存中的程序转存在外存储器

4.2 存储管理

- 实存储管理

- 4) 交换技术

- 需要时，将待执行的作业重新载入内存
 - 执行时调入内存、不执行时放入外存 -->
 - 称这种技术为：“滚入滚出”技术。
 - 问题：作业“滚入滚出”，需花费大量CPU时间
 - 这一问题的解决导致虚拟存储技术的出现。
 - 在实时、分时、以及批处理系统中均有应用

4.2 存储管理

- 虚拟存储管理

- “**实存**”管理的特点是作业必须全部装入一个连续的空间，否则作业无法运行
- “**虚存**”是通过内外存结合，用软件的方法实现存储器的扩充，以实现大作业的运行。参见Windows中的设置...

4.2 存储管理

- 虚拟存储管理

- **虚拟存储器**，提供给用户一个比实际内存大得多的存储空间，用户不用再考虑存储空间不够用的情况

- **基本概念：**

- 虚拟地址：程序访问的逻辑地址
 - 实际地址：处理器可直接访问的主存地址
 - 虚拟地址空间：虚拟地址的集合
 - 实际地址空间：计算机主存
 - **程序和数据所在的虚拟地址，必须放入主存的实际地址中才能执行**

4.2 存储管理

- 虚拟存储管理

- 如何进行虚拟地址空间 与 实际地址空间 之间的转换

- 通过 动态地址映像机构 来完成

- 按需把 虚拟地址空间 调入到 实际地址空间

- 虚拟地址空间的限制——不可能无限大

- 指令中地址长度的限制，32 bits

- 外存容量的限制。

4.2 存储管理

- 虚拟存储管理

- 虚拟存储要解决问题

- 何时把作业程序需要的那部分内容装入内存
 - 放在实际地址空间的什么位置
 - 实际地址空间不够用时，把哪部分程序淘汰出内存

- 常用虚拟存储技术

- 分页存储管理
 - 分段存储管理
 - 段页存储管理

4.2 存储管理

- 虚拟存储管理 -- 分页存储管理
 - 虚拟地址空间和实际地址空间划分为大小相同的页(page)
 - 以页为单位把内存分配给各作业程序
 - 分配给一个作业程序的页之间可以不连续
 - 作业程序无须一次都装入内存，而是按需装入

4.2 存储管理

- 虚拟存储管理 -- 分页存储管理

- 虚拟地址的表示:

- 用一个数对 (p, d) : p 为页号, d 为页内相对地址

- 页的大小: 2 的幂 (2^n)

- 页的大小 = 512 bytes
 - 地址 $(1200)_{10}$ 相当于 $(2, 176)_{10}$
 - 如用八进制表示, 则 $(512)_{10} \rightarrow (1000)_8$
 - 地址 $(1200)_{10} \rightarrow (2260)_8$ 相当于 $(2, 260)_8$
 - (便于计算页号和相对地址)

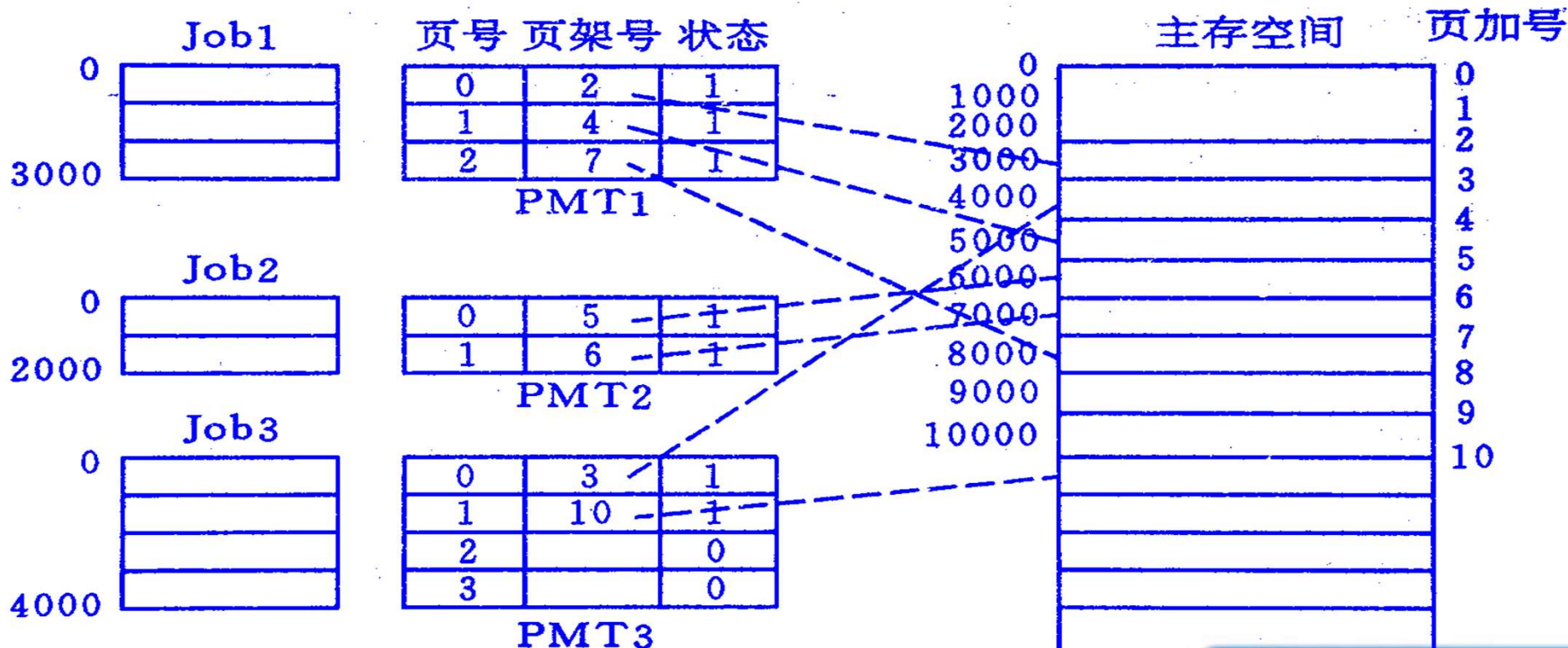
4.2 存储管理

• 虚拟存储管理 -- 分页存储管理

– 页表，页面映像表 (PMT) *

- 页号：作业程序的页面序号，从 0 开始计数
- 页架号：内存中的实际地址空间的页面序号
- 状态：“1” 在内存内，“0” 不在内存内

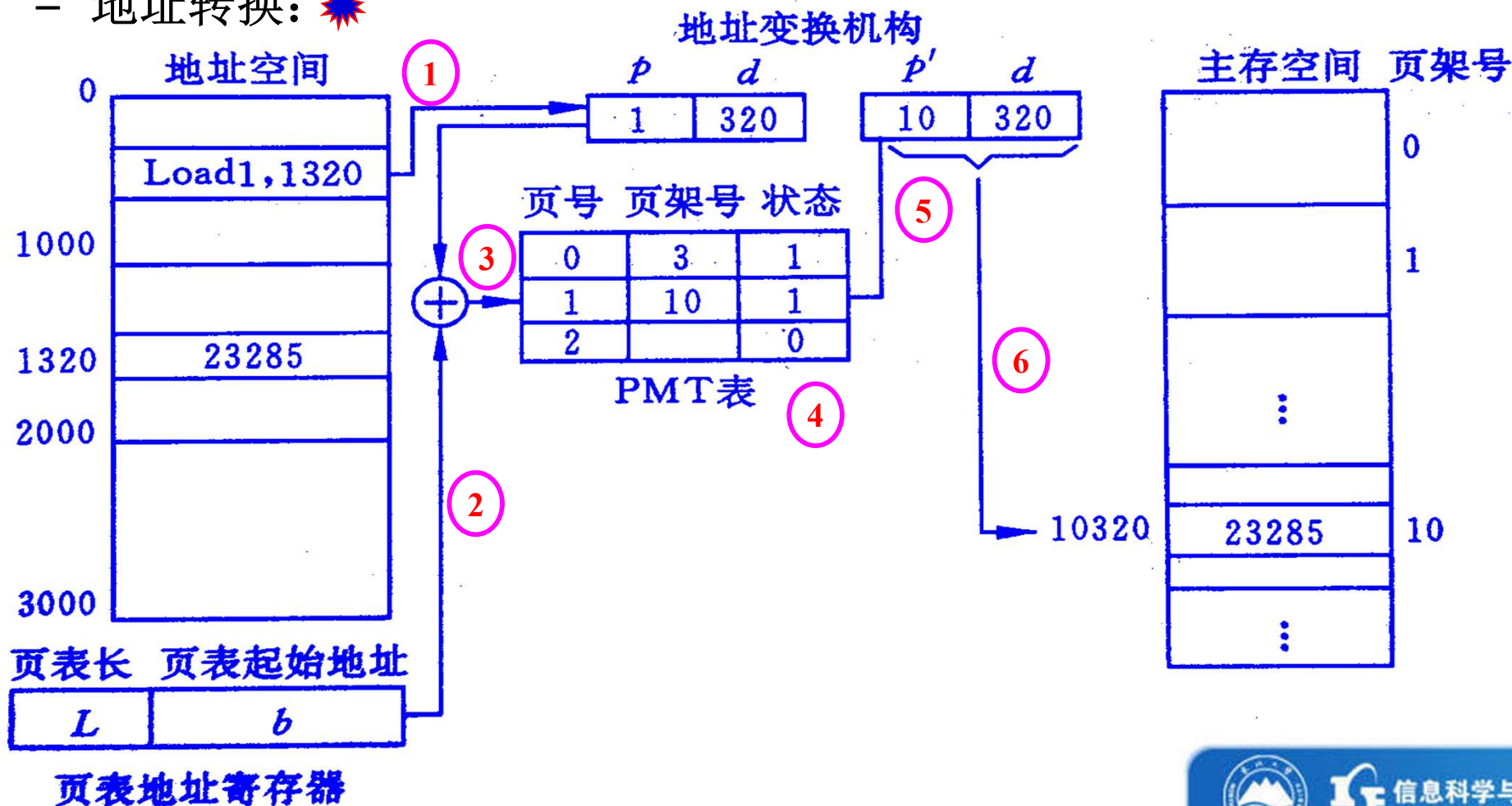
当作业调入内存时，OS为每个作业建立一个PMT，用于判断某页面是否在内存？在内存的哪一页中？



• 虚拟存储管理 -- 分页

- 页表地址寄存器
始地址 b ，当
。用于地址转换
- 地址转换：✱

- ① 当CPU执行一条访问内存的指令，如：Load1,1320 时，地址变换机构把逻辑地址分解成（页号，页内相对地址）两部分（ p, d ）；
- ② 从页表地址寄存器中获取页表的首地址；
- ③ 在页表中，查找相应的页号 p
- ④ 若页面状态为“0”不在内存，系统产生缺页中断，由存储管理模块将其调入内存。
- ⑤ 将页表中的页架号 p' ，送给地址变换机构
- ⑥ 地址变换机构根据 p', d ，形成实际地址



4.2 存储管理

• 虚拟存储管理 -- 分页存储管理

– 页面更换算法，页面淘汰算法

• 先进先出 (FIFO)

– 例如：一个用户程序，共有5个页面

要执行外存中的新页面时，必须将内存中的页面放入外存，即进行页面更换。更换不当，会出现抖动，从而降低系统效率

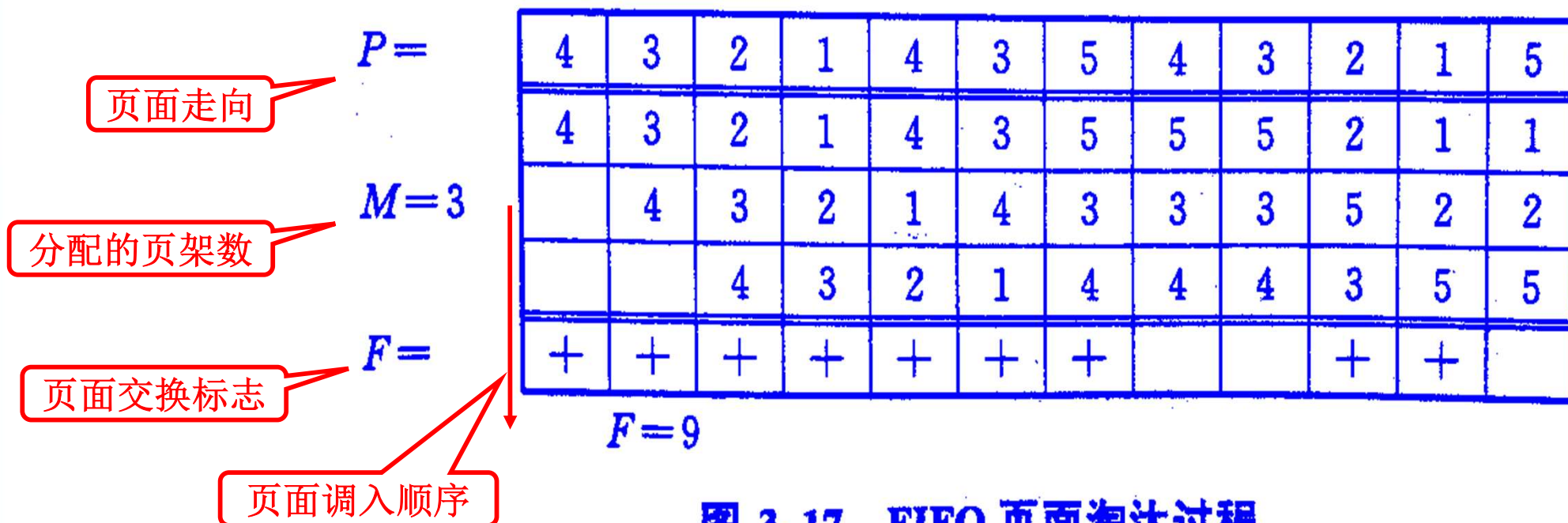


图 3.17 FIFO 页面淘汰过程

$$\text{缺页中断率 } f = \frac{F}{\text{页面走向总数}} = \frac{9}{12} = 75\%$$

4.2 存储管理

• 虚拟存储管理 —— 分页存储管理

- 页面更换算法，页面淘汰算法

• 最近最少使用算法 (LRU, Least Recently Used)

- 需要记录每个页面自上次访问到现在的时间，但开销太大
- 常用近似算法，在页架中加入“引用位”来标识使用情况。实现过程

- ① 将页架用链指针链成循环链表；
- ② 循环查找引用位；
- ③ 如为0，则进行置换，置换后标记引用位为1
- ④ 如为1，则将其置为0，然后继续查找



页架号 页号 引用位 指针

0			
1			
2	4	0	•
3			
4	2	1	•
5			
6	5	0	•
7	1	0	•
8			

(a)

替换指针

q

页架号 页号 引用位 指针

0			
1			
2	4	0	•
3			
4	2	1	•
5			
6	6	1	•
7	1	0	•
8			

(b)



4.2 存储管理

- 虚拟存储管理 -- 分页存储管理
 - 存储保护 -
 - 用页表表长(L)来控制访问内存地址的范围
 - 如果合法，才进行地址转换
 - 如页号超出页表长，则产生越界中断

4.2 存储管理

- 虚拟存储管理 —— 分页存储管理

- 优点:

- 不要求作业在内存中连续，较好地解决了碎片问题
 - 作业地址空间不受内存限制，为用户提供足够大的空间，便于多道程序作业

- 缺点:

- 要求一定的硬件支持，增加了成本
 - 因增加页表及管理程序，增加了内存开销
 - 处理页面交换需要占用CPU时间

4.2 存储管理

- 虚拟存储管理 -- 分段存储管理
 - 把程序按模块划分为几个部分
 - 每一部分属于一个段 (segment)
 - 为每个段在内存分配一块连续分区
 - 各段之间不要求连续
 - 暂时不运行的段可以不调入内存

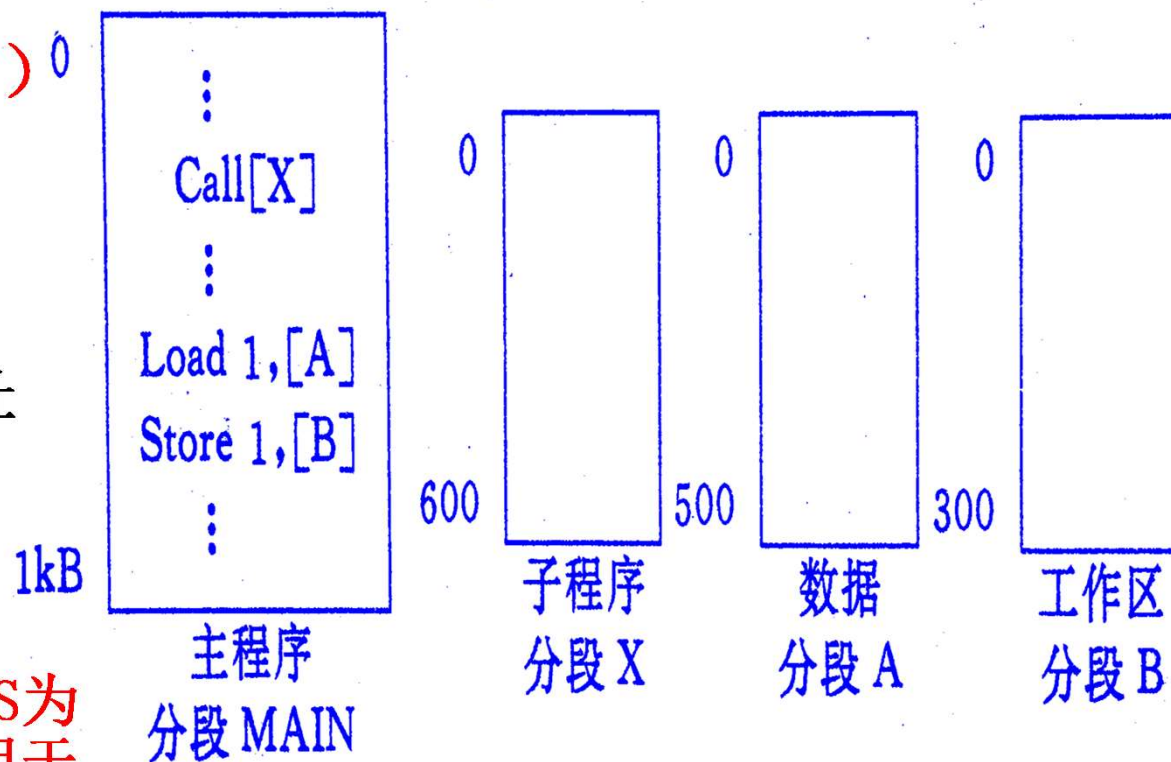
4.2 存储管理

• 虚拟存储管理 —— 分段存储管理

- **地址结构**：每个虚拟地址由两部分构成 (s, w) ， s 为段号， w 为段内地址

- **段表，段映像表 (SMT)** ⁰

- 段号
- 段的长度
- 段在主存中的起始地址
- 段的状态
- (存取权限)
- 当作业调入内存时，OS为作业创建一个段表，用于保存作业中的所有段的信息。作业完成后撤消段表



4.2 存储管理

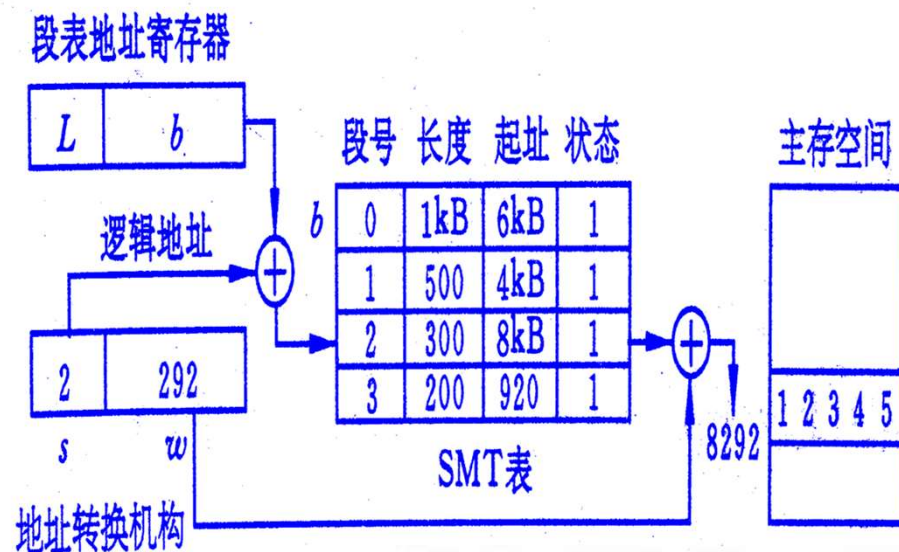
- 虚拟存储管理 -- 分段存储管理

- **段表地址寄存器**（段表长度 L ，段表起始地址 b ），用于地址转换

- 先由硬件地址转换机构和段表地址寄存器得到段表的位置
- 再在所在的段中找到指定的实际存储空间

- 存储保护

- 通过设置段表的存取权限来控制
- 用段表表长来控制



4.2 存储管理

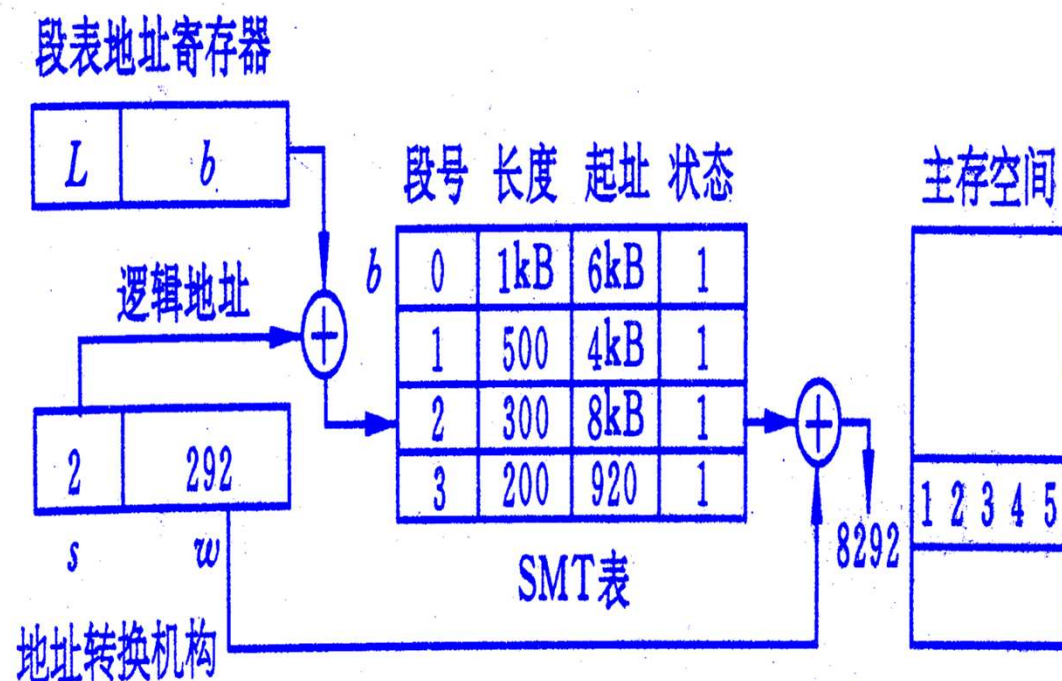
• 虚拟存储管理 —— 分段存储管理

- 分段管理的优点

- 便于程序模块化处理
- 便于处理变化的数据
- 便于共享分段

- 分段管理的缺点

- 要增加硬件（寄存器）
- CPU花费时间进行地址转换
- 分段大小受到主存的限制
- 灵活性较分页管理差



4.2 存储管理

- 虚拟存储管理 -- 段页式存储管理
 - 分页和分段表存储管理的结合
 - 参见 **P125**

End

